International Journal of Scientific Research & Engineering Trends
Volume 9, Issue 6, Nov-Dec-2023, ISSN (Online): 2395-566X

Infrastructure as Code: Puppet and Ansible Co-Deployment in Hybrid Environments

Felix Corvin
Dream Logic Labs

Abstract- In the modern era of digital infrastructure, organizations are increasingly adopting Infrastructure as Code (IaC) to manage and automate the provisioning and configuration of resources across both on-premises and cloud environments. IaC ensures consistency, repeatability, and efficiency by allowing infrastructure to be defined and maintained through version-controlled code. Among the many tools available, Puppet and Ansible have emerged as two of the most widely adopted solutions, each bringing distinct advantages to the automation landscape. Puppet is based on a declarative model and is particularly suited for policy enforcement and large-scale system state management. Ansible, by contrast, follows a procedural model and is known for its flexibility, simplicity, and agentless operation. This review examines the rationale, architecture, and best practices behind co-deploying Puppet and Ansible within hybrid environments. Rather than viewing these tools as mutually exclusive, the paper explores how they can be used in complementary roles to achieve higher degrees of automation maturity, compliance, and infrastructure resilience. The review discusses how Puppet can handle base operating system configurations and enforce long-term system states, while Ansible is better suited for orchestration tasks, application deployment, and change management.

Index Terms- Infrastructure as Code (IaC); Puppet; Ansible; Hybrid Cloud; Configuration Management; Automation; DevOps; Compliance as Code; GitOps; Secrets Management; Policy Enforcement; Declarative Configuration; Procedural Automation; CI/CD Integration; Multi-Cloud Orchestration.

I. INTRODUCTION

1. Background and Motivation

As enterprises continue to adopt hybrid infrastructure strategies, encompassing a mix of traditional on-premises systems and modern cloud platforms, the need for consistent and automated management has become more urgent. The complexity of maintaining configuration consistency, enforcing compliance, and enabling rapid deployment in such environments has driven the adoption of Infrastructure as Code. This approach transforms the management of servers, networks, and applications into repeatable, testable, and version-controlled processes, improving agility while reducing human error.

Among the many Infrastructure as Code tools available, Puppet and Ansible have established themselves as leading solutions. Puppet employs a declarative model that is ideal for managing the desired state of systems over time, using agents and a centralized server model to enforce policies across fleets of machines. It is often favored in large-scale enterprise environments where regulatory compliance and long-term system state are priorities. In contrast, Ansible uses a procedural approach, executing step-by-step instructions to perform tasks. Its agentless architecture, based on SSH or APIs, makes it easy to adopt and highly flexible for a wide range of use cases, from server provisioning to complex application deployment workflows.

These contrasting characteristics mean that while Puppet and Ansible can function independently, they can also be co-deployed to cover different layers and responsibilities within a hybrid infrastructure. Each tool can complement the other, with Puppet managing foundational configuration and compliance enforcement, and Ansible orchestrating services, performing ad-hoc changes, and responding to events. This dual approach offers organizations a more granular, adaptable, and resilient automation framework.

2. Objectives of the Review

The primary objective of this review is to evaluate the strategic and technical considerations of deploying Puppet and Ansible together in hybrid environments. The paper aims to explore how the unique strengths of each tool can be harnessed in a coordinated manner to improve infrastructure consistency, security, and operational agility. It provides a comprehensive overview of integration models, including side-by-side deployment and layered automation strategies,

© 2023 IJSRET

along with best practices for inventory management, secrets handling, and role separation.

This review also identifies common challenges encountered in co-deployment scenarios, such as overlapping responsibilities, skill gaps, and scalability issues. Through real-world use cases and industry examples, the paper illustrates how these challenges can be addressed to enable successful deployments. Additionally, it highlights current trends and future directions, such as the use of artificial intelligence for intelligent automation, the shift toward compliance-as-code, and the application of Infrastructure as Code principles in container-native and serverless environments. By presenting both the conceptual framework and practical insights, this review serves as a guide for IT architects and DevOps practitioners looking to build cohesive and future-ready automation strategies using Puppet and Ansible.

II. FOUNDATIONS OF PUPPET AND ANSIBLE

1. Puppet: Declarative State Management

Puppet is a configuration management tool built on a declarative paradigm, meaning the user defines the desired end-state of the system, and Puppet ensures that the system converges to and maintains that state. It operates through an agent-server model, where managed nodes (agents) regularly check in with a central Puppet master to retrieve updated configuration catalogs compiled based on predefined manifests. Puppet's Domain-Specific Language (DSL) provides high-level abstraction to describe infrastructure components such as packages, services, and files. This approach allows Puppet to enforce consistent states across thousands of machines with minimal human intervention, making it well-suited for enterprise environments that require strict adherence to compliance baselines and long-term configuration stability. The idempotency inherent in Puppet's architecture ensures that configurations are applied consistently, even after multiple runs, without causing unintended changes.

2. Ansible: Procedural Task Automation

In contrast, Ansible follows a procedural model where infrastructure is managed through a series of explicitly defined tasks. These tasks are written in YAML-based playbooks, making Ansible easy to learn and read. Ansible executes commands over SSH or APIs, eliminating the need for persistent agents on target nodes. This agentless architecture simplifies deployment and maintenance, particularly in heterogeneous environments. While Ansible is capable of enforcing state to a certain degree, it excels in orchestration, provisioning, application deployment, and event-driven automation. Its modular structure, with reusable roles and inventory files, enables teams to manage dynamic environments with speed and agility. The ability to perform ad-hoc commands and integrate with APIs makes Ansible particularly powerful for real-time operations and service-level tasks.

3. Comparative Strengths and Use Cases

Puppet and Ansible serve overlapping yet distinct purposes in the automation stack. Puppet is advantageous for systems that require long-term consistency, compliance enforcement, and auditability. It integrates well with policy-as-code frameworks and is often the tool of choice for base OS provisioning and infrastructure-wide configurations. Ansible, meanwhile, is highly flexible and excels in use cases where procedural logic, conditional execution, and real-time orchestration are required. It is particularly useful for application deployments, database provisioning, and rolling updates across services. When deployed together, Puppet can maintain the foundational system configuration, while Ansible provides fine-grained control over workflows and deployments layered on top. This division of responsibilities minimizes redundancy and maximizes automation coverage across diverse infrastructure types.

III. HYBRID INFRASTRUCTURE MANAGEMENT WITH IAC

1. The Nature of Hybrid Environments

Hybrid infrastructure refers to the coexistence of on-premises datacenters, private cloud frameworks, and public cloud platforms within a single operational model. This architectural approach provides flexibility, scalability, and cost optimization but introduces significant complexity in managing infrastructure consistently. Systems across different environments often use distinct provisioning methods, network configurations, identity management systems, and security policies. Ensuring a uniform and secure configuration across these diverse components becomes challenging, especially as infrastructure scales and becomes more dynamic.

2. IaC Benefits in Hybrid Contexts

Infrastructure as Code offers a unified approach to managing hybrid infrastructure by codifying configurations and deployment logic into reusable, testable, and version-controlled artifacts. This enables infrastructure to be provisioned and modified in a controlled and repeatable manner, regardless of the underlying environment. IaC allows for rapid scaling, faster recovery, and improved compliance by maintaining configuration baselines that can be audited and enforced programmatically. Version control systems provide traceability of changes, while integration with CI/CD pipelines enables continuous delivery of infrastructure changes. These capabilities are particularly beneficial in hybrid environments where consistency and speed are

© 2023 IJSRET

essential to maintaining service availability and security across varied platforms.

3. Challenges in IaC Across Platforms

Despite its advantages, implementing IaC across hybrid platforms presents several challenges. Differences in operating systems, cloud provider APIs, and infrastructure abstractions can complicate module reuse and configuration portability. Secrets management becomes more difficult when sensitive credentials must be securely shared across disjointed systems. Network configurations, firewalls, and routing rules vary significantly between on-premises and cloud environments, requiring custom automation logic to maintain connectivity and security. Drift detection identifying when a system deviates from its intended state—is also more complex when state is distributed across environments. Addressing these issues requires careful tool selection, architectural planning, and consistent enforcement of automation standards across the entire infrastructure footprint.

IV. CO-DEPLOYMENT ARCHITECTURE: PUPPET + ANSIBLE

1. Architectural Integration Models

The co-deployment of Puppet and Ansible can follow several architectural models depending on the organizational goals and infrastructure topology. One common model involves deploying Puppet as the baseline configuration manager, responsible for OS-level configurations, security hardening, and compliance enforcement. Ansible is then layered on top as an orchestration tool for handling application-level deployments, task-based updates, and integration workflows. Another model treats the two tools as parallel systems with domain-specific responsibilities, where each team or environment uses the most appropriate tool for its tasks. Integration can be facilitated through Git repositories, where both Puppet manifests and Ansible playbooks are stored and managed together, enabling teams to adopt GitOps practices.

2. Communication, Inventory, and Role Separation

Successful co-deployment requires a clear separation of roles and careful management of inventories. Puppet typically uses its own resource catalogs and node classification mechanisms, while Ansible relies on dynamic or static inventory files. To avoid conflicts, it is important to designate ownership of specific configuration domains. For example, Puppet may be assigned responsibility for firewall rules, user accounts, and kernel parameters, while Ansible handles database configurations and service-level operations. Shared variables, secrets, and facts must be carefully synchronized using external tools such as Hiera for Puppet and Ansible Vault or dynamic inventories for Ansible. Clear documentation and governance processes help reduce overlap and ensure smooth coexistence.

3. Automation Pipelines and GitOps Alignment

Modern infrastructure practices increasingly rely on automation pipelines and GitOps principles, where infrastructure code is stored in version control systems and changes are automatically propagated to target environments. Puppet and Ansible can both be integrated into these workflows using tools like Jenkins, GitHub Actions, or GitLab CI. Puppet manifests can be tested and applied using r10k or Code Manager, while Ansible playbooks can be triggered via webhooks or pipeline stages. GitOps alignment ensures that infrastructure changes are peer-reviewed, traceable, and deployable in a consistent manner. By managing both tools from a unified repository and enforcing tagging, branching, and approval workflows, organizations can build robust, policy-driven automation frameworks that span their entire hybrid estate.

V. REAL-WORLD USE CASES

1. Telecom: Multi-Vendor Device Automation

In the telecommunications sector, automation must span a diverse array of hardware and software platforms. Puppet is often employed to enforce base configurations across heterogeneous server farms, including firmware settings, SNMP configurations, and OS-level hardening policies. Its model-driven architecture ensures that systems remain compliant over time, even after manual interventions. Ansible complements this by handling complex orchestrations involving network switches, firewalls, and load balancers from multiple vendors. Since Ansible supports command-line interface interactions and REST APIs, it can easily automate vendor-specific configurations in real-time. In this model, Puppet maintains the core infrastructure state, while Ansible manages workflow automation, patch orchestration, and firmware upgrades, resulting in improved operational consistency across distributed telco environments.

2. Finance: Audit-Driven Compliance and Deployment

Financial institutions are subject to stringent compliance standards such as PCI-DSS and SOX, which require rigorous control and visibility over infrastructure. Puppet is frequently used to baseline server configurations, enforce cryptographic policies, and ensure audit logging is active and persistent. Ansible is used to execute tasks such as secure application deployment, encryption key rotation, and failover testing, which need to occur dynamically during release windows. In such deployments, change requests are linked with version-controlled Puppet modules and Ansible playbooks, forming a CI/CD pipeline governed by approval processes. The combination ensures a clear separation between state management and operational flexibility, improving auditability while maintaining deployment agility.

© 2023 IJSRET

3. Government and Defense: Offline Infrastructure Automation

Government and defense environments often operate in air-gapped or highly restricted networks where access to public repositories or real-time internet services is prohibited. In such settings, Puppet is deployed with internally mirrored content from Puppet Forge, while Ansible playbooks are curated and maintained through secure internal Git repositories. Satellite servers are used to manage RHEL subscriptions, content views, and Puppet configurations. Ansible is run manually or through scheduled job execution to handle dynamic operations such as backup validation, log aggregation, and patch application. The co-deployment of Puppet and Ansible in these secure enclaves supports resilient infrastructure management while meeting strict compliance, audit, and access control requirements.

VI. MONITORING, DRIFT MANAGEMENT, AND REPORTING

1. Configuration Drift Detection and Remediation

Configuration drift the divergence of actual infrastructure state from its defined configuration can result in security vulnerabilities, performance degradation, and compliance violations. Puppet's architecture provides continuous drift detection, where each agent periodically compares its local system state with the catalog provided by the Puppet master. Deviations are automatically corrected, and changes are logged for audit purposes. Ansible, while not inherently designed for state enforcement, can perform ad-hoc scans using the check flag to identify deviations without making changes. Combining both tools allows organizations to detect, report, and remediate drift across both static and dynamic components of the environment, ensuring long-term consistency and reliability.

2. Telemetry and Logging Integration

To ensure observability across the automation landscape, both Puppet and Ansible support integration with logging and telemetry systems. PuppetDB serves as a central repository for facts, reports, and resources, enabling analytics and visualization of configuration changes over time. Ansible provides JSON-formatted logs and can be extended with callback plugins to send execution data to external logging systems. Integration with ELK (Elasticsearch, Logstash, Kibana), Prometheus, or Splunk enables infrastructure teams to correlate automation events with system performance metrics and security incidents. This visibility helps troubleshoot failures, audit user activity, and monitor compliance status in real-time.

3. Compliance Reporting and Auditing

In regulated industries, demonstrating infrastructure compliance is not just a best practice—it is a legal requirement. Puppet supports compliance reporting through its Enterprise Console and integrations with tools like OpenSCAP, which perform vulnerability scans and generate regulatory compliance reports. Ansible, when combined with roles such as ansible-lockdown, can be used to verify configuration compliance with industry standards such as CIS Benchmarks. Reports from both tools can be exported in machine-readable formats or pushed to central audit platforms. The combination of continuous enforcement from Puppet and on-demand auditing from Ansible provides a comprehensive compliance assurance framework that is both preventive and detective.

VII. SECURITY AND SECRETS MANAGEMENT

1. Credential and Secrets Handling

Managing secrets securely is one of the most critical aspects of any infrastructure automation strategy. Puppet typically integrates with Hiera, a hierarchical configuration system that can be backed by secrets managers such as HashiCorp Vault. This enables encrypted data to be injected into manifests without hardcoding credentials. Ansible uses its own solution called Ansible Vault, which encrypts variables and files using symmetric keys. For dynamic secrets, Ansible supports plugins for integrating with external secret providers such as AWS Secrets Manager and Azure Key Vault. When used together, both tools must adhere to consistent secrets management policies to prevent duplication or leakage, especially in shared environments.

2. Role-Based Access Control (RBAC)

Controlling who can run automation tasks and on which systems is essential for maintaining infrastructure integrity. Puppet Enterprise includes a role-based access control model that allows administrators to define user permissions by node groups, environments, and actions. Ansible Automation Platform (AAP) also includes RBAC through its web interface and REST API, enabling fine-grained access control over playbooks, inventories, and credentials. In hybrid deployments, aligning these RBAC models with central identity providers such as LDAP or Active Directory improves manageability and reduces risk. Defining clear separation of duties between Puppet and Ansible operators also ensures accountability and minimizes cross-contamination of responsibilities.

3. Secure Communication in Hybrid Topologies

In hybrid and multi-region environments, maintaining secure communication between automation components and managed nodes is a top priority. Puppet uses TLS with certificate-based authentication to secure traffic between masters and agents. Ansible, being agentless, relies on SSH or API credentials, which must be secured using bastion hosts or

© 2023 IJSRET

VPN tunnels in distributed deployments. Both tools should use encrypted transports, least-privilege credentials, and audit logging to ensure secure operations. In environments with varying trust levels—such as public cloud, partner networks, or DMZs additional layers of security such as jump hosts, firewalls, and encrypted vaults must be integrated into the automation workflow to maintain data confidentiality and integrity.

VIII. PERFORMANCE AND SCALABILITY

1. Puppet Server and Agent Performance

Puppet's performance in large-scale deployments depends on efficient catalog compilation, agent scheduling, and the capacity of the Puppet server infrastructure. The Puppet master compiles catalogs for each node based on their defined classes and environments, which can create bottlenecks when managing thousands of nodes. To mitigate this, organizations employ multiple compilers, scale out their Puppet servers, and use environments like Code Manager and r10k for parallel catalog delivery. Fine-tuning factors such as agent run intervals, fact generation, and environment segmentation ensures that Puppet continues to scale without overloading compute resources. Additionally, Puppet's compiled catalog caching and transaction reporting mechanisms allow for high visibility while maintaining performance under large workloads.

2. Ansible Execution and Job Scaling

Ansible's agentless design provides simplicity but presents unique scalability challenges. The default mechanism for parallelism forks limits how many hosts can be managed simultaneously. In enterprise deployments, increasing the fork count and optimizing SSH transport parameters significantly boosts Ansible's throughput. The introduction of Execution Environments in Ansible Automation Platform (AAP) further supports scalable task execution using containerized playbook runtimes. These isolated environments ensure consistent behavior across hosts and pipelines. The use of dynamic inventories also reduces resource overhead by targeting only relevant hosts, enhancing execution efficiency in environments with fluctuating topology or workloads.

3. Scalability in Multi-Cloud and Multi-Tenant Deployments

In multi-cloud and multi-tenant environments, scalability extends beyond performance to include governance, modularity, and environment isolation. Puppet achieves this through role-based classification, node groups, and environment-specific hierarchies in Hiera. It supports federated deployments where multiple Puppet masters manage distinct trust domains. Ansible scales through the use of tagged playbooks, inventory groups, and variable scoping. Both tools support integration with service catalogs and self-service portals, enabling Infrastructure as Code consumption across development, staging, and production environments. To achieve true horizontal scalability, enterprises combine containerization, distributed job schedulers, and CI/CD pipeline integration with well-architected IaC structures and role definitions.

IX. TOOLCHAIN ECOSYSTEM AND EXTENSIBILITY

1. Puppet Forge and Ansible Galaxy

The strength of any automation platform lies in its ecosystem, and both Puppet and Ansible benefit from extensive community and vendor-supported content hubs. Puppet Forge offers thousands of reusable modules for operating systems, middleware, databases, and security baselines. These modules conform to best practices and reduce the effort needed to onboard new systems or enforce policies. Ansible Galaxy provides a similar function for roles and collections, which are curated YAML playbooks and plugins packaged for reuse. Many roles on Galaxy are maintained by vendors such as Red Hat, Cisco, and AWS, ensuring compatibility and lifecycle support. These repositories accelerate automation implementation and foster community-driven improvement.

2. Plugin, Callback, and API Extensions

Puppet and Ansible both offer robust extensibility through plugins and APIs. Puppet supports custom facts, types, and providers, enabling administrators to extend its capability to manage complex or non-standard resources. Its REST APIs allow integration with CMDBs, ticketing systems, and monitoring tools. Ansible, on the other hand, features callback plugins, inventory plugins, and connection plugins that facilitate deep integration with external systems and support asynchronous event handling. The Ansible Automation Platform also includes APIs for job control, reporting, and inventory synchronization. These extensibility options enable both tools to integrate seamlessly into broader enterprise toolchains and support emerging DevOps and SRE workflows.

3. Integration with DevOps and ITSM Tools

Effective Infrastructure as Code practices rely on seamless integration with DevOps pipelines and IT service management platforms. Puppet and Ansible both support integration with tools like Jenkins, GitHub Actions, GitLab CI, and Azure DevOps for continuous integration and deployment. These integrations enable automated infrastructure testing, approval workflows, and release gating. For ITSM alignment, both tools can connect to ServiceNow and Jira, enabling infrastructure changes to be linked to change tickets and compliance audits. Terraform, another IaC tool focused on provisioning, is often used alongside Puppet and Ansible to handle infrastructure creation, with configuration and orchestration delegated to Puppet and Ansible respectively.

© 2023 IJSRET

This integration ecosystem ensures that IaC aligns with both development velocity and governance requirements.

Challenges and Lessons Learned

Operational Complexity in Co-Deployment

Deploying Puppet and Ansible together introduces operational complexity that must be carefully managed. Without clear role definitions, teams may encounter redundancy, conflicting changes, or duplicated effort. For instance, having both tools manage the same system users or firewall rules can lead to race conditions and state inconsistency. Establishing a well-defined boundary—such as using Puppet for base OS and security settings and Ansible for application deployment—is crucial. Documentation, modular design, and tagging conventions help reduce complexity. Additionally, maintaining separate pipelines, inventories, and secrets stores for each tool can complicate CI/CD workflows, necessitating governance and orchestration layers to unify operations.

Organizational Skill Gaps

The dual-tool model requires staff to be proficient in both Puppet's declarative language and Ansible's procedural playbook logic. This presents a skill gap, especially in environments where DevOps maturity is still evolving. Teams accustomed to one tool may resist adopting the other, citing learning curves or operational overhead. Training, documentation, and internal knowledge-sharing play an essential role in bridging this gap. Organizations that invest in role-based training, pair programming, and mentoring initiatives can accelerate adoption and minimize tool-specific silos. Tool standardization, code reviews, and shared repositories also improve team coherence and reduce the risk of automation fragmentation.

Best Practice Patterns and Anti-Patterns

Co-deployment success hinges on following well-established patterns and avoiding known anti-patterns. A best practice is to use Puppet for baseline system enforcement and Ansible for overlay tasks, ensuring that both tools work on distinct layers of the stack. Another proven strategy is to centralize variables and secrets in a shared system to avoid duplication or leakage. Common anti-patterns include allowing overlapping control of the same resources, hardcoding configuration data into playbooks or manifests, and skipping testing stages before deployment. By adhering to modular design, enforcing tagging, and treating infrastructure code with the same rigor as application code, organizations can avoid common pitfalls and build sustainable automation frameworks.

Future Directions

AI-Assisted Automation

As infrastructure complexity grows, the next wave of innovation lies in AI-assisted automation. Both Puppet and Ansible are increasingly integrating with machine learning tools that can anticipate configuration drifts, suggest corrective actions, or automatically remediate faults. For instance, anomaly detection algorithms can parse system telemetry and alert administrators when configurations deviate from historical norms. Future implementations may allow AI to auto-generate Ansible playbooks or Puppet manifests based on intent or observed patterns. This level of intelligence could drastically reduce manual intervention and accelerate incident resolution, especially in large-scale environments with high deployment velocity. Additionally, AI-enabled root cause analysis could further streamline the feedback loop in CI/CD pipelines by detecting misconfigurations earlier in the lifecycle.

Policy as Code and Compliance-as-Code

The shift toward codifying security and compliance policies often referred to as policy as code is another important development. Integrations with Open Policy Agent (OPA), InSpec, and compliance-as-code toolchains are emerging in both Puppet and Ansible ecosystems. These tools allow organizations to define compliance requirements declaratively and validate them automatically against infrastructure states. In regulated industries, this approach enhances auditability and ensures continuous enforcement of governance frameworks such as NIST, CIS, and GDPR. Puppet's integration with OpenSCAP and Ansible's compliance roles are early steps in this direction, but more advanced features, such as real-time policy injection and compliance testing within CI pipelines, are expected to become standard in future IaC toolchains.

IaC in Containerized and Serverless Environments

As workloads shift toward Kubernetes, containers, and serverless platforms, traditional IaC tools are evolving to support ephemeral infrastructure. Puppet has introduced capabilities in Kubernetes configuration through projects like Nebula and Bolt, while Ansible is extending its reach with Kubernetes modules and OpenShift-native playbooks. These extensions allow automation to address not just bare-metal or virtual infrastructure, but also dynamic workloads orchestrated by container runtimes. Similarly, Function-as-a-Service (FaaS) platforms require lightweight, event-driven automation that can provision functions and manage service bindings without persistent state. The future of IaC will involve tighter coupling with these paradigms, requiring Puppet and Ansible to adapt their models for distributed, short-lived, and cloud-native resources.

X. CONCLUSION

This review has explored the co-deployment of Puppet and Ansible as a strategic approach to managing hybrid IT environments through Infrastructure as Code. While Puppet brings the rigor of declarative, policy-driven configuration management, Ansible complements it with its flexibility, simplicity, and procedural task execution. Together, they form

© 2023 IJSRET

a robust automation framework capable of addressing both long-term infrastructure consistency and dynamic orchestration requirements. By delineating responsibilities—Puppet for enforcing system baselines and Ansible for managing application layers and service workflows—organizations can reduce operational overhead, enhance compliance, and respond more quickly to changing business needs.

The review highlighted real-world use cases across telecom, finance, and government sectors, illustrating how these tools can operate in tandem across cloud, on-premises, and air-gapped environments. It also examined essential areas such as secrets management, RBAC, drift detection, logging, scalability, and extensibility. Challenges such as operational complexity and skill gaps were acknowledged, along with strategies to mitigate them through modular design, training, and best practice patterns.

Looking forward, trends such as AI-driven automation, policy as code, and support for containerized platforms will continue to shape the evolution of Infrastructure as Code. Organizations that adopt a layered automation strategy—leveraging the strengths of both Puppet and Ansible will be better positioned to build resilient, scalable, and compliant hybrid infrastructure systems that meet both current demands and future growth.

REFERENCES

1. Qiao, L., Bo, J., Guo, L., Hu, F., Hu, N., & Ran, R. (2020). Tools of Centralized Management Platform Based on Puppet and Ansible. IOP Conference Series: Materials Science and Engineering, 750.
2. Hinkel, G., Groenda, H., Vannucci, L., Denninger, O., Cauli, N., & Ulbrich, S. (2015). A Domain-Specific Language (DSL) for Integrating Neuronal Networks in Robot Control. Proceedings of the 2015 Joint MORSE/VAO Workshop on Model-Driven Robot Software Engineering and View-based Software-Engineering.
3. Atkisson, B.J. (2019). Pulling the Puppet Strings with Ansible.
4. Battula, V. (2022). Legacy systems, modern solutions: A roadmap for UNIX administrators. Royal book publishers
5. Staff, V. (2016). Release: Ansible Tower 3 by Red Hat.
6. Lolong, S., & Kistijantoro, A.I. (2011). Domain Specific Language (DSL) development for desktop-based database application generator. Proceedings of the 2011 International Conference on Electrical Engineering and Informatics, 1-6.
7. Contreras, F.R., Álvarez, B., Sánchez, P., & Pastor, J.A. (2016). U-DSL: a Domain Specific Language for Ubiquitous Systems. IEEE Latin America Transactions, 14, 4416-4420.
8. Wilkinson, M.J., Roda, F., Walter, G.M., James, M.E., Nipper, R.W., Walsh, J., Allen, S.L., North, H.L., Beveridge, C.A., & Ortiz-Barrientos, D. (2019). Adaptive divergence in shoot gravitropism creates hybrid sterility in an Australian wildflower. Proceedings of the National Academy of Sciences of the United States of America, 118.
9. Rossi, E.D., Baloch, Z.W., Pusztaszeri, M.P., & Faquin, W.C. (2018). The Milan System for Reporting Salivary Gland Cytopathology (MSRSGC): An ASC-IAC-Sponsored System for Reporting Salivary Gland Fine-Needle Aspiration. Acta Cytologica, 62, 157 - 165.
10. Hochgeschwender, N., Biggs, G., & Voos, H. (2018). A Reference Architecture for Deploying Component-Based Robot Software and Comparison with Existing Tools. 2018 Second IEEE International Conference on Robotic Computing (IRC), 121-128.
11. Wang, R., Li, C., & Gu, R. (2016). Problem statement of SDN and NFV co-deployment in cloud datacenters.
12. Gao, Z., Karlsson, I., Geisen, S., Kowalchuk, G.A., & Jousset, A. (2019). Protists: Puppet Masters of the Rhizosphere Microbiome. Trends in plant science, 24 2, 165-176 .
13. Breitbart, M., Bonnain, C., Malki, K., & Sawaya, N.A. (2018). Phage puppet masters of the marine microbial realm. Nature Microbiology, 3, 754 - 766.
14. Akshay, N. (2019). Multi-Vendor Remote Monitoring and Control System for Hydraulic Drive Systems.
15. Kazim, M., Liu, L., & Zhu, S.Y. (2018). A Framework for Orchestrating Secure and Dynamic Access of IoT Services in Multi-Cloud Environments. IEEE Access, 6, 58619-58633.
16. Ploennigs, J., Lehmann, M., Wollschlaeger, B., Mai, T.L., & Kabitzsch, K. (2014). Multi-objective device selection approach for component-based automation systems. Proceedings of the 2014 IEEE Emerging Technology and Factory Automation (ETFA), 1-8.
17. Comley, A.R. (1990). EXPERIENCE IN THE INTEGRATION OF TELEMETRY, AUTOMATION AND DATA LOGGING SYSTEMS.
18. Horton, E., & Parnin, C. (2019). V2: Fast Detection of Configuration Drift in Python. 2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE), 477-488.
19. Mahajan, S., Khandelwal, S., & K, M. (2020). Security and Privacy in Multi-Cloud Environments. International Journal of Psychosocial Rehabilitation.
20. Mittal, P., & Mehta, P. (2020). Optimization of Software Development Process by Plugin Integration with Jira – A Project Management Tool in Devops. Industrial & Manufacturing Engineering eJournal.
21. Battula, V. (2021). Dynamic resource allocation in Solaris/Linux hybrid environments using real-time

© 2023 IJSRET

monitoring and AI-based load balancing. International Journal of Engineering Technology Research & Management, 5 (11), 81–89. https://ijetrm.com/

22. Maria, T.J., Dizon, G.R., Esquivel, V.A., Deja, J.A., & Chua, U.C. (2020). Designing Grit: Discovering Features Towards Supporting Novice Programmer DevOps Integration. Proceedings of the 2020 Symposium on Emerging Research from Asia and on Asian Contexts and Cultures.

© 2023 IJSRET